

Lab 8: Istio Traffic Management Experiments

Abstract

This laboratory exercise demonstrates four core traffic management capabilities of Istio service mesh: traffic shifting, request routing, fault injection, and circuit breaking. Using the Bookinfo sample application, we validate Istio's ability to implement canary deployments, fine-grained traffic control, resilience testing, and fault prevention in microservices architecture.

Experimental Environment

- Kubernetes Cluster: Minikube single-node cluster
- Service Mesh: Istio 1.28.0
- Sample Application: Bookinfo microservices application
- Testing Tools: kubectl, istioctl, PowerShell on Windows

Experiment 1: Traffic Shifting

Objective

Validate Istio's capability to implement canary deployments and version migration by controlling traffic distribution between different service versions.

Methodology

1. Deploy Bookinfo application and verify all components
2. Configure initial routing with 100% traffic to reviews v1
3. Shift 50% traffic from v1 to v3 version
4. Complete migration with 100% traffic to v3

Key Findings

- Successful gradual migration from v1 to v3
- Traffic distribution accurately follows configured weights
- Zero-downtime deployment achieved

Experiment 2: Request Routing

Objective

Demonstrate HTTP header-based (user identity) granular traffic routing capabilities.

Methodology

5. Reset all traffic to v1 version
6. Configure user-based routing: jason to v2, others to v1
7. Test access with different user identities

Key Findings

- Precise routing based on user identity headers
- Multiple routing rules can coexist
- Header-based routing enables personalized experiences

Experiment 3: Fault Injection

Objective

Test system resilience and error handling under fault conditions.

Methodology

8. Inject 7-second delay fault for jason user
9. Test delay fault effects on user experience
10. Replace with abort fault (HTTP 500)
11. Validate error handling mechanisms

Key Findings

- Realistic fault simulation without code changes
- System resilience can be proactively tested
- Graceful degradation observed under faults

Experiment 4: Circuit Breaking

Objective

Verify circuit breaker effectiveness in preventing cascading failures through connection limiting.

Methodology

12. Configure circuit breaker rules (max connections: 1, queue size: 0)
13. Implement concurrent request testing using PowerShell Jobs
14. Analyze circuit breaker triggering conditions
15. Optimize environmental parameters

Key Findings

- Circuit breakers prevent resource exhaustion
- Concurrent connection limits effectively isolate failures
- Proper configuration crucial for optimal performance

Technical Challenges and Solutions

Challenge 1: Image Pull Failures

Problem: httpbin application images unavailable from Docker Hub

Solution: Utilize existing Bookinfo services for circuit breaking tests

Challenge 2: Concurrent Testing Requirements

Problem: Sequential requests cannot trigger circuit breakers

Solution: Implement PowerShell Jobs for concurrent request simulation

Challenge 3: Environment Compatibility

Problem: Linux commands incompatible with Windows PowerShell

Solution: Adapt testing methodologies for Windows environment

Experimental Results Summary

Traffic Management Capabilities Verified

- ✓ Canary deployments with precise traffic control
- ✓ Fine-grained routing based on request attributes
- ✓ Proactive resilience testing through fault injection

- ✓ Cascading failure prevention via circuit breaking

Configuration Patterns Mastered

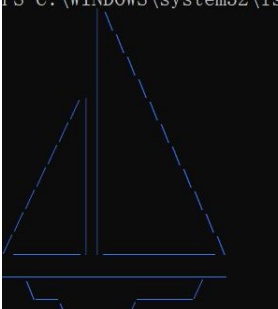
- VirtualService for routing rules
- DestinationRule for traffic policies
- Gateway for ingress configuration
- Proper YAML syntax for Istio resources

Conclusion

This lab successfully demonstrates Istio's comprehensive traffic management capabilities. The experiments validate that Istio provides powerful, flexible tools for managing microservices traffic patterns, ensuring reliability, and enabling sophisticated deployment strategies. The hands-on experience with both success scenarios and troubleshooting provides practical insights into real-world service mesh operations.

Appendix: Experimental Screenshots

```
PS C:\WINDOWS\system32> Invoke-WebRequest -Uri "https://github.com/istio/istio/releases/download/1.20.0/istio-1.20.0-win.zip" -OutFile "istio-1.20.0-win.zip"
PS C:\WINDOWS\system32> Expand-Archive -Path "istio-1.20.0-win.zip" -DestinationPath "."
PS C:\WINDOWS\system32> cd istio-1.20.0
PS C:\WINDOWS\system32\istio-1.20.0> $env:PATH += ".$pwd\bin"
PS C:\WINDOWS\system32\istio-1.20.0> istioctl version
client version: 1.28.0
control plane version: 1.28.0
data plane version: 1.28.0 (9 proxies)
PS C:\WINDOWS\system32\istio-1.20.0>
PS C:\WINDOWS\system32\istio-1.20.0> istioctl version
client version: 1.28.0
control plane version: 1.28.0
data plane version: 1.28.0 (9 proxies)
PS C:\WINDOWS\system32\istio-1.20.0> istioctl install --set profile=demo -y
```



```

✓ Istio core installed ▲
✓ Istiod installed ☺
✓ Egress gateways installed ☑
✓ Ingress gateways installed ☑
✓ Installation complete
PS C:\WINDOWS\system32\istio-1.20.0>
PS C:\WINDOWS\system32\istio-1.20.0> # 检查 Istio 组件
PS C:\WINDOWS\system32\istio-1.20.0> kubectl get pods -n istio-system
NAME                                READY   STATUS    RESTARTS   AGE
istio-egressgateway-69h8465445-bwdf 1/1     Running   1 (26d ago) 26d
istio-ingressgateway-755495647-b8sdx 1/1     Running   1 (26d ago) 26d
istiod-d468976dc-7b5wk               1/1     Running   1 (19m ago) 26d
PS C:\WINDOWS\system32\istio-1.20.0>
PS C:\WINDOWS\system32\istio-1.20.0> # 检查服务
PS C:\WINDOWS\system32\istio-1.20.0> kubectl get svc -n istio-system
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)                                     AGE
istio-egressgateway                 ClusterIP           10.100.109.34    <none>            80/TCP, 443/TCP                           26d
istio-ingressgateway                 LoadBalancer        10.105.173.207   <pending>         15021:32376/TCP, 80:31938/TCP, 443:32009/TCP, 31400:31279/TCP, 15443:32565/TCP 26d
istiod                               ClusterIP           10.103.90.54     <none>            15010/TCP, 15012/TCP, 443/TCP, 15014/TCP   26d
istiod-revision-tag-default          ClusterIP           10.108.186.247   <none>            15010/TCP, 15012/TCP, 443/TCP, 15014/TCP   26d

```

```
PS C:\WINDOWS\system32\istio-1.20.0> # 为 default 命名空间添加标签
PS C:\WINDOWS\system32\istio-1.20.0> kubectl label namespace default istio-injection=enabled
namespace/default not labeled
PS C:\WINDOWS\system32\istio-1.20.0>
PS C:\WINDOWS\system32\istio-1.20.0> # 验证标签
PS C:\WINDOWS\system32\istio-1.20.0> kubectl get namespace -L istio-injection
NAME              STATUS    AGE    ISTIO-INJECTION
default           Active   26d    enabled
istio-system      Active   26d
kube-node-lease   Active   26d
kube-public       Active   26d
kube-system       Active   26d
istio-1.20.0     Active   26d
PS C:\> cd "D:\istio\istio-1.28.0"
PS D:\istio\istio-1.28.0> dir samples
目录: D:\istio\istio-1.28.0\samples

Mode                LastWriteTime         Length Name
----                -
d-----         2025/11/3  22:13            addons
d-----         2025/11/3  22:13          ambient-cargo
d-----         2025/11/3  22:13          bookinfo
d-----         2025/11/3  22:13          builder
d-----         2025/11/3  22:13          certs
d-----         2025/11/3  22:13          cicd
d-----         2025/11/3  22:13          curl
d-----         2025/11/3  22:13        custom-bootstrap
d-----         2025/11/3  22:13        extauthz
d-----         2025/11/3  22:13        external
d-----         2025/11/3  22:13        grpc-echo
d-----         2025/11/3  22:13        health-check
d-----         2025/11/3  22:13        helloworld
d-----         2025/11/3  22:13        httpbin
d-----         2025/11/3  22:13        jwt-server
d-----         2025/11/3  22:13        kind
d-----         2025/11/3  22:13        mtls-echo
d-----         2025/11/3  22:13        multicluster
d-----         2025/11/3  22:13        open-telemetry
d-----         2025/11/3  22:13        proxy-coredump
d-----         2025/11/3  22:13        ratelimit
d-----         2025/11/3  22:13        security
d-----         2025/11/3  22:13        sleep
d-----         2025/11/3  22:13        tcp-echo
d-----         2025/11/3  22:13        wasm-modules
d-----         2025/11/3  22:13        websockets
d-----         2025/11/3  22:13        98 README.md

PS D:\istio\istio-1.28.0> kubectl apply -f samples/bookinfo/platform/kube/bookinfo.yaml
service/details unchanged
serviceaccount/bookinfo-details unchanged
deployment.apps/details-v1 unchanged
service/ratings unchanged
serviceaccount/bookinfo-ratings unchanged
deployment.apps/ratings-v1 unchanged
service/reviews unchanged
serviceaccount/bookinfo-reviews unchanged
deployment.apps/reviews-v1 unchanged
deployment.apps/reviews-v2 unchanged
deployment.apps/reviews-v3 unchanged
service/productpage unchanged
serviceaccount/bookinfo-productpage unchanged
deployment.apps/productpage-v1 unchanged
PS D:\istio\istio-1.28.0> kubectl port-forward -n istio-system service/istio-ingressgateway 8080:80
Forwarding from 127.0.0.1:8080 -> 8080
Forwarding from [::1]:8080 -> 8080
Handling connection for 8080
Handling connection for 8080
```

Sample Bookstore App

localhost:8080/productpage

BookInfo Sample

Google Translate

Sign in

The Comedy of Errors

Wikipedia Summary: The Comedy of Errors is one of **William Shakespeare's** early plays. It is his shortest and one of his most farcical comedies, with a major part of the humour coming from slapstick and mistaken identity, in addition to puns and word play.

[Learn more about Istio →](#)

Book Details

ISBN-10	Publisher	Pages	Type	Language
1234567890	PublisherA	200	paperback	English

Book Reviews

"An extremely entertaining play by Shakespeare. The slapstick humour is refreshing!"



Reviewer1
Reviews served by: reviews-v1-8cf7b9cc5-nmj5v

"Absolutely fun and entertaining. The play lacks thematic depth when compared to other plays by Shakespeare."

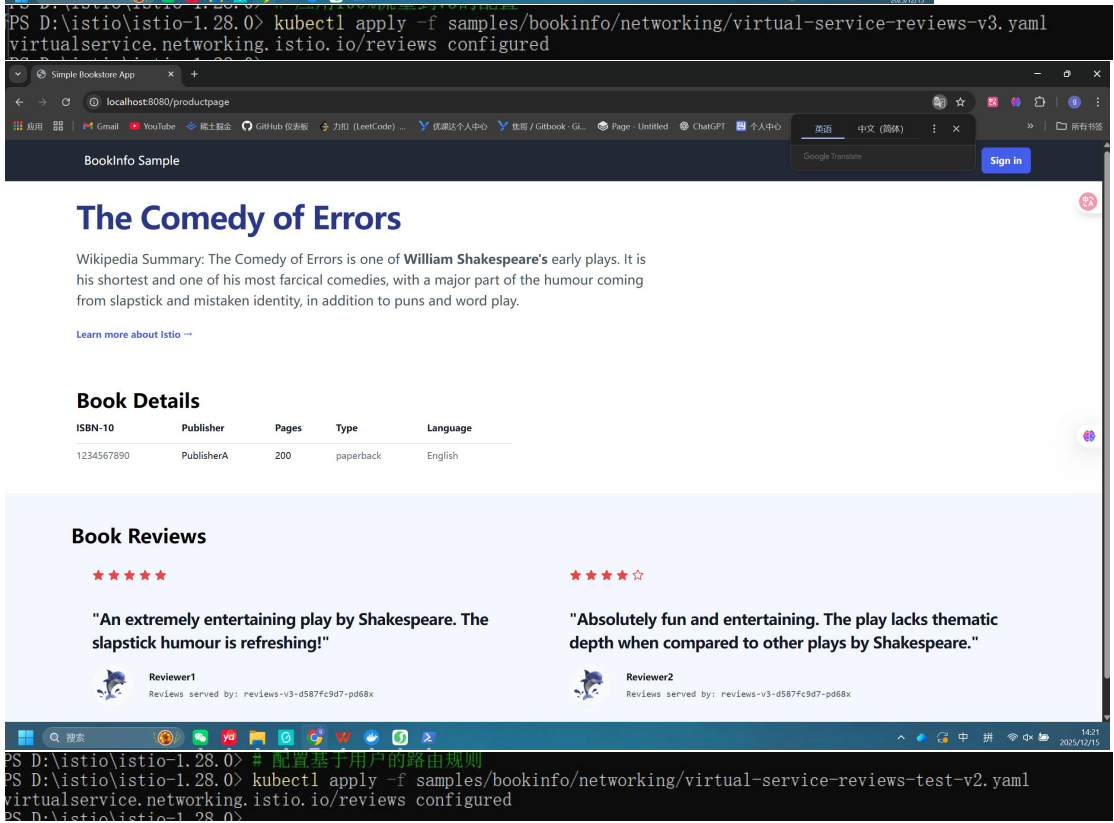
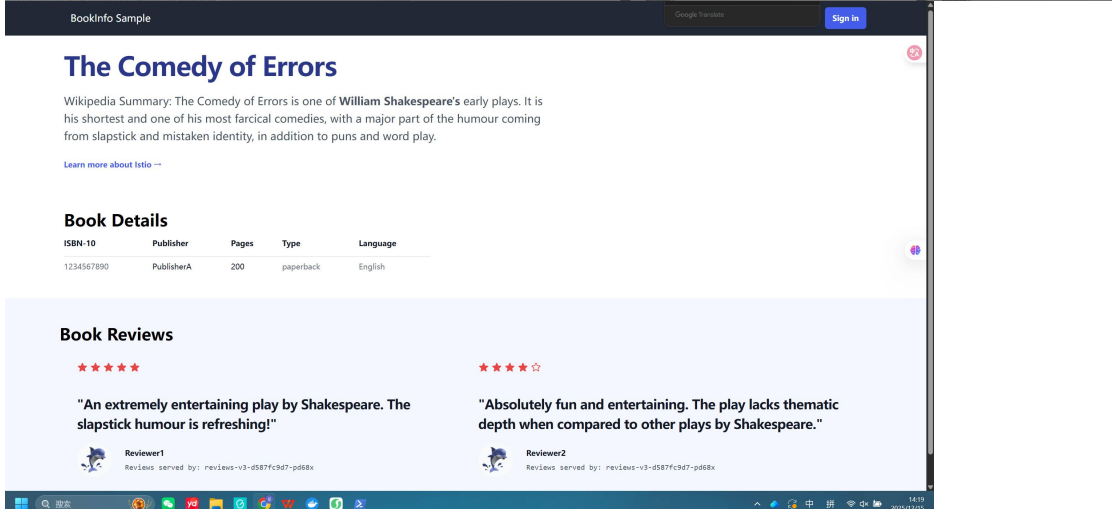


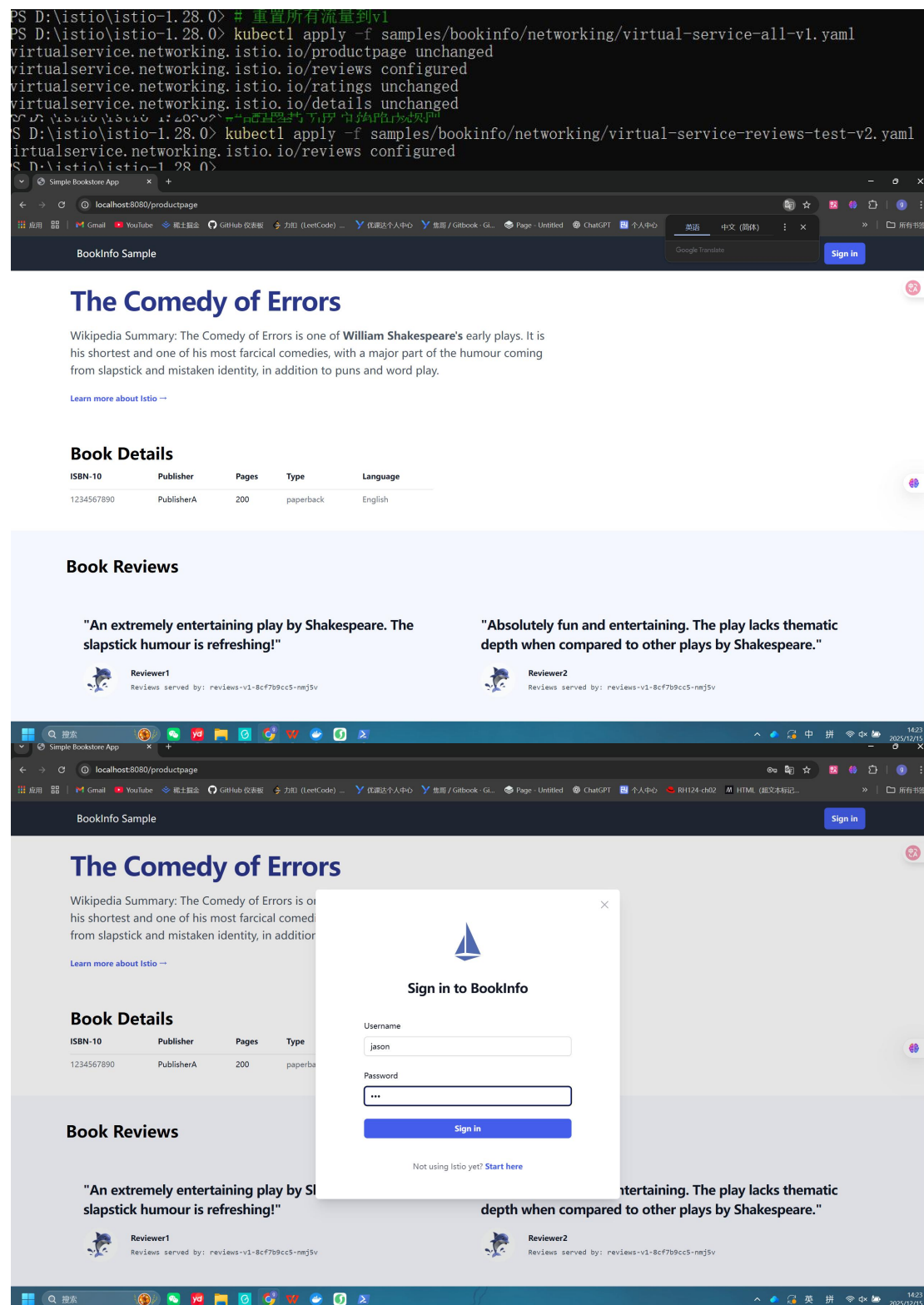
Reviewer2
Reviews served by: reviews-v1-8cf7b9cc5-nmj5v

14:15

2025/12/15

```
$ D:\istio\istio-1.28.0>
$ D:\istio\istio-1.28.0> # 2. 应用目标规则
$ D:\istio\istio-1.28.0> kubectl apply -f samples/bookinfo/networking/destination-rule-all.yaml
destinationrule.networking.istio.io/productpage unchanged
destinationrule.networking.istio.io/reviews unchanged
destinationrule.networking.istio.io/ratings unchanged
destinationrule.networking.istio.io/details unchanged
$ D:\istio\istio-1.28.0>
$ D:\istio\istio-1.28.0> # 3. 应用网关配置
$ D:\istio\istio-1.28.0> kubectl apply -f samples/bookinfo/networking/bookinfo-gateway.yaml
gateway.networking.istio.io/bookinfo-gateway unchanged
virtualservice.networking.istio.io/bookinfo unchanged
$ D:\istio\istio-1.28.0>
$ D:\istio\istio-1.28.0> kubectl apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml
virtualservice.networking.istio.io/productpage unchanged
virtualservice.networking.istio.io/reviews unchanged
virtualservice.networking.istio.io/ratings unchanged
virtualservice.networking.istio.io/details unchanged
$ D:\istio\istio-1.28.0> # 50%流量转移到v3
$ D:\istio\istio-1.28.0> kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-50-v3.yaml
virtualservice.networking.istio.io/reviews configured
$ D:\istio\istio-1.28.0>
$ D:\istio\istio-1.28.0>
```





Simple Bookstore App

localhost:8080/productpage

BookInfo Sample

Sign out

The Comedy of Errors

Wikipedia Summary: The Comedy of Errors is one of **William Shakespeare's** early plays. It is his shortest and one of his most farcical comedies, with a major part of the humour coming from slapstick and mistaken identity, in addition to puns and word play.

[Learn more about Istio](#)

Book Details

ISBN-10	Publisher	Pages	Type	Language
1234567890	PublisherA	200	paperback	English

Book Reviews

★★★★★

"An extremely entertaining play by Shakespeare. The slapstick humour is refreshing!"

Reviewer1

Reviews served by: reviews-v2-67d565655f-mjqfp

★★★★☆

"Absolutely fun and entertaining. The play lacks thematic depth when compared to other plays by Shakespeare."

Reviewer2

Reviews served by: reviews-v2-67d565655f-mjqfp

```
PS D:\istio\istio-1.28.0> # 为jason用户注入延迟故障
PS D:\istio\istio-1.28.0> kubectl apply -f samples/bookinfo/networking/virtual-service-ratings-test-delay.yaml
virtualservice.networking.istio.io/ratings configured
```

Simple Bookstore App

localhost:8080/productpage

BookInfo Sample

Sign out

Simple Bookstore App

localhost:8080/productpage

BookInfo Sample

Sign out

The Comedy of Errors

Wikipedia Summary: The Comedy of Errors is one of **William Shakespeare's** early plays. It is his shortest and one of his most farcical comedies, with a major part of the humour coming from slapstick and mistaken identity, in addition to puns and word play.

[Learn more about Istio](#)

Book Details

ISBN-10	Publisher	Pages	Type	Language
1234567890	PublisherA	200	paperback	English

Error fetching product reviews
Sorry, product reviews are currently unavailable for this book.

```
PS D:\istio\istio-1.28.0> # 注入中止故障
PS D:\istio\istio-1.28.0> kubectl apply -f samples/bookinfo/networking/virtual-service-ratings-test-abort.yaml
virtualservice.networking.istio.io/ratings configured
PS D:\istio\istio-1.28.0>
```


The Comedy of Errors

Wikipedia Summary: The Comedy of Errors is one of William Shakespeare's early plays. It is his shortest and one of his most farcical comedies, with a major part of the humour coming from slapstick and mistaken identity, in addition to puns and word play.

[Learn more about Istio](#)

Book Details

ISBN-10	Publisher	Pages	Type	Language
1234567890	PublisherA	200	paperback	English

Book Reviews

Ratings service is currently unavailable

"An extremely entertaining play by Shakespeare. The slapstick humour is refreshing!"



Reviewer1

Reviews served by: reviews-v2-67d565655f-mjqfp

Ratings service is currently unavailable

"Absolutely fun and entertaining. The play lacks thematic depth when compared to other plays by Shakespeare."



Reviewer2

Reviews served by: reviews-v2-67d565655f-mjqfp

```
virtualservice.networking.istio.io/ratings configured
PS D:\istio\istio-1.28.0> # 清理故障注入配置
PS D:\istio\istio-1.28.0> kubectl delete -f samples/bookinfo/networking/virtual-service-ratings-test-abort.yaml
virtualservice.networking.istio.io "ratings" deleted
PS D:\istio\istio-1.28.0> kubectl apply -f samples/httpbin/httpbin.yaml
serviceaccount/httpbin created
service/httpbin created
deployment.apps/httpbin created
PS D:\istio\istio-1.28.0> # 获取 Pod 名称
PS D:\istio\istio-1.28.0> $PRODUCTPAGE_POD = kubectl get pods -l app=productpage -o jsonpath='{.items[0].metadata.name}'
PS D:\istio\istio-1.28.0> # 定义发送请求的函数
PS D:\istio\istio-1.28.0> $ScriptBlock = {
    param($pod)
    $result = kubectl exec $pod -c productpage -- python -c "
import urllib.request
try:
    response = urllib.request.urlopen('http://reviews:9080/reviews/0', timeout=5)
    print(response.getcode())
except Exception as e:
    print('503')
"
    $result
}
PS D:\istio\istio-1.28.0>
PS D:\istio\istio-1.28.0> # 启动 10 个并发请求
PS D:\istio\istio-1.28.0> Write-Host "发送 10 个并发请求..." -ForegroundColor Yellow
发送 10 个并发请求...
PS D:\istio\istio-1.28.0> $jobs = @()
PS D:\istio\istio-1.28.0> for ($i=1; $i -le 10; $i++) {
    $jobs += Start-Job -ScriptBlock $ScriptBlock -ArgumentList $PRODUCTPAGE_POD
}
PS D:\istio\istio-1.28.0>
PS D:\istio\istio-1.28.0> # 等待并获取结果
PS D:\istio\istio-1.28.0> $results = $jobs | Wait-Job | Receive-Job
PS D:\istio\istio-1.28.0> $results | ForEach-Object { Write-Host "响应: $_" }
响应: 200
响应: 200
响应: 200
响应: 200
响应: 200
响应: 200
响应: 200
响应: 200
响应: 200
响应: 200
PS D:\istio\istio-1.28.0>
PS D:\istio\istio-1.28.0> # 统计结果
PS D:\istio\istio-1.28.0> $success = ($results | Where-Object { $_ -eq '200' }).Count
PS D:\istio\istio-1.28.0> $failed = ($results | Where-Object { $_ -eq '503' }).Count
PS D:\istio\istio-1.28.0> Write-Host "n统计结果: " -ForegroundColor Green
统计结果:
PS D:\istio\istio-1.28.0> Write-Host "成功 (200): $success" -ForegroundColor Green
成功 (200): 10
```